

# Java Programming

## Questions & Solutions

### Q1. Compare 2D Points for Distance from Origin

Write a program to check distance of 2D points from origin and print the point with highest distance. Create class Point with attributes double x, double y. In Solution class define main method to read values for three point objects. Create method pointWithHighestOriginDistance(Point p1, Point p2, Point p3) that returns the point with highest distance from origin.

**Sample Input:** 2 2 / 3 3 / -4 -4   **Output:** -4.0 -4.0

```
import java.util.Scanner;

class Point {
    double x;
    double y;
}

class Solution {
    public static Point pointWithHighestOriginDistance(Point p1, Point p2, Point p3) {
        double d1 = Math.sqrt(p1.x * p1.x + p1.y * p1.y);
        double d2 = Math.sqrt(p2.x * p2.x + p2.y * p2.y);
        double d3 = Math.sqrt(p3.x * p3.x + p3.y * p3.y);

        if (d1 >= d2 && d1 >= d3) return p1;
        else if (d2 >= d1 && d2 >= d3) return p2;
        else return p3;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Point p1 = new Point(); p1.x = sc.nextDouble(); p1.y = sc.nextDouble();
        Point p2 = new Point(); p2.x = sc.nextDouble(); p2.y = sc.nextDouble();
        Point p3 = new Point(); p3.x = sc.nextDouble(); p3.y = sc.nextDouble();

        Point result = pointWithHighestOriginDistance(p1, p2, p3);
        System.out.println(result.x);
        System.out.println(result.y);
    }
}
```

### Q2. Find Minimum Character in a String

Read a string and find the character with the minimum ASCII value (smallest character). Print that character.

**Sample Input:** Hello   **Output:** H

```
import java.util.*;

public class Solution {
    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);
```

```
String s = sc.next();

char min = s.charAt(0);
for (int i = 1; i < s.length(); i++) {
    if (s.charAt(i) < min) {
        min = s.charAt(i);
    }
}
System.out.println(min);
}
```

### Q3. Factorials of Input Numbers

Write a program to read 5 numbers and print factorials of each. Final answers should be non-decimal numbers (use long to avoid overflow).

**Sample Input:** 2 3 4 6 5    **Output:** 2 / 6 / 24 / 720 / 120

```
import java.util.*;

public class Solution {
    public static long fact(long num) {
        if (num == 0 || num == 1) return 1;
        long facto = 1;
        for (long i = 1; i <= num; i++) {
            facto = facto * i;
        }
        return facto;
    }

    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);
        long num1 = sc.nextLong(); long num2 = sc.nextLong();
        long num3 = sc.nextLong(); long num4 = sc.nextLong();
        long num5 = sc.nextLong();

        System.out.println(fact(num1));
        System.out.println(fact(num2));
        System.out.println(fact(num3));
        System.out.println(fact(num4));
        System.out.println(fact(num5));
    }
}
```

#### Q4. Find Second Largest Prime Number

Write a program to read n numbers and display the second largest prime number. The first number entered is n. Read n numbers and find the second largest prime among them.

**Sample Input:** 6 / 1 3 17 33 44 43    **Output:** 17

```
import java.util.*;

public class Solution {

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        List<Integer> primes = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
            if (isPrime(num)) primes.add(num);
        }

        Collections.sort(primes, Collections.reverseOrder());
        System.out.println(primes.get(primes.size() - 2));
    }
}
```

#### Q5. Find Documents with Odd Pages (TCS Xplore Java4)

Create class Document with attributes: id (int), title (String), folderName (String), pages (int). Write getters, setters and parameterized constructor. Implement static method docsWithOddPages in Solution class which takes an array of Document objects and returns documents with odd number of pages, sorted in ascending order of id.

**Sample Input:** id=1,title=resume,folder=personal,pages=50 | id=2,title=question1,folder=exams,pages=55 | id=3,title=question2,folder=exams,pages=45 | id=4,...    **Output:** 2 question1 exams 55 / 3 question2 exams 45

```
import java.util.*;

class Document {
    int id;
    String title;
    String folderName;
    int pages;

    Document(int id, String title, String folderName, int pages) {
        this.id = id;
        this.title = title;
        this.folderName = folderName;
    }
}
```

```
this.pages = pages;
}
}

public class Solution {

    public static Document[] docsWithOddPages(Document[] docs) {
        List<Document> oddList = new ArrayList<>();
        for (Document d : docs) {
            if (d.pages % 2 != 0) oddList.add(d);
        }
        oddList.sort((a, b) -> a.id - b.id);
        return oddList.toArray(new Document[0]);
    }

    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);

        Document[] docs = new Document[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            String title = sc.next();
            String folder = sc.next();
            int pages = sc.nextInt();
            docs[i] = new Document(id, title, folder, pages);
        }

        Document[] result = docsWithOddPages(docs);
        for (Document d : result) {
            System.out.println(d.id + " " + d.title + " " + d.folderName + " " + d.pages);
        }
    }
}
```

## Q6. Sort Books by Price (TCS Xplore Java3)

Create class Book with attributes: id (int), title (String), author (String), price (double). Write getters, setters and parameterized constructor. Implement static method sortBooksByPrice in Solution class which takes an array of Book objects and returns the array sorted in ascending order of price. No two books will have the same price.

**Sample Input:** 1 hello writer1 50 / 2 cup writer2 55 / 3 Planet writer3 45 / 4 India ... **Output:** Books sorted by price ascending

```
import java.util.*;

class Book {
    int id;
    String title;
    String author;
    double price;

    Book(int id, String title, String author, double price) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.price = price;
    }
}

public class Solution {

    public static Book[] sortBooksByPrice(Book[] books) {
        Arrays.sort(books, (a, b) -> Double.compare(a.price, b.price));
        return books;
    }

    public static void main(String args[]) throws Exception {
        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            String title = sc.next();
            String author = sc.next();
            double price = sc.nextDouble();
            books[i] = new Book(id, title, author, price);
        }

        Book[] result = sortBooksByPrice(books);
        for (Book b : result) {
            System.out.println(b.id + " " + b.title + " " + b.author + " " + b.price);
        }
    }
}
```